

Detecção de Danos em Carros com Deep Learning

Cauê Eberspacher Menezes

Laboratório UNIFESP - Campus Talim

YOLOv8 - Faster R-CNN - Mask R-CNN

OBJETIVO DO PROJETO

Dado uma foto de um carro, o modelo identifica automaticamente onde estão os amassados — sem inspeção manual.

Seguradoras	Locadoras	Oficinas
Vistoria automática de sinistros	Detecção de danos na devolução de veículos	Triagem rápida para orçamento de reparos

A comparação entre três métodos permite identificar o mais adequado para esta tarefa.

BASE DE DADOS

COCO Car Damage

59

imagens de treino

11

imagens de validação

~140

anotações de dano

1

categoria: damage

Formato: COCO JSON + JPG · 1024×1024 px

CarDD

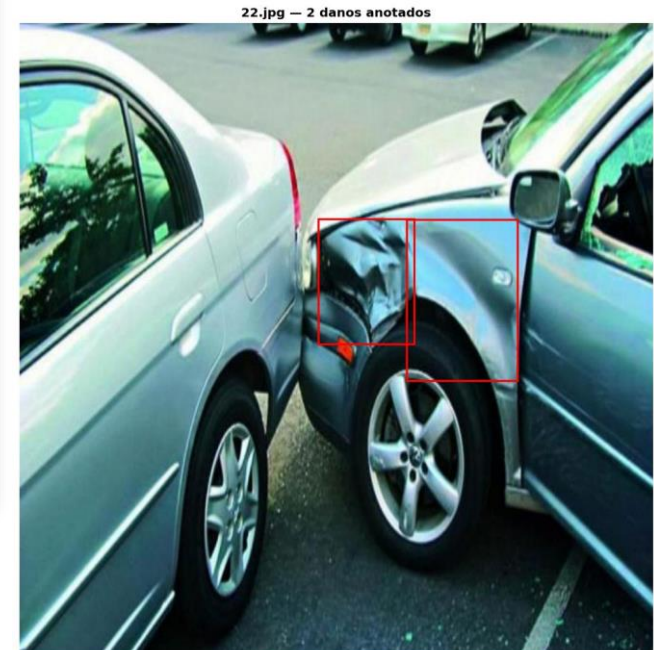
4.000

imagens disponíveis

Dataset maior, mais adequado para treino robusto.

Limitação de hardware impediu o uso neste trabalho, mas os modelos foram projetados para suportá-lo.

Formato: COCO JSON + JPG



OS 3 MODELOS DE DEEP LEARNING

YOLOv8

Uma etapa

- Analisa a imagem toda de uma vez.
- Extremamente rápido (~72ms/imagem).
- Ideal para aplicações em tempo real.
- Modelo usado: YOLOv8n (nano).
- 3 milhões de parâmetros.

Faster R-CNN

Duas etapas

- Propõe regiões suspeitas (RPN).
- Classifica cada região separadamente.
- Backbone ResNet50 pré-treinado.
- Mais preciso que o YOLO.
- Mais lento, porém mais robusto.

Mask R-CNN

Deteção + Segmentação

- Extensão do Faster R-CNN.
- Aprende o contorno exato do dano.
- Usa as máscaras do dataset COCO.
- Backbone ResNet50 pré-treinado.
- Maior compreensão visual do dano.

APRENDIZADO DE REDE

Hiperparâmetros

lr (taxa de aprendizado)	0.005
momentum	0.9
weight decay	0.0005
épocas	15 (50 no YOLOv8)
batch size	2 a 4
conf threshold	0.5

Gradiente e deformação do espaço

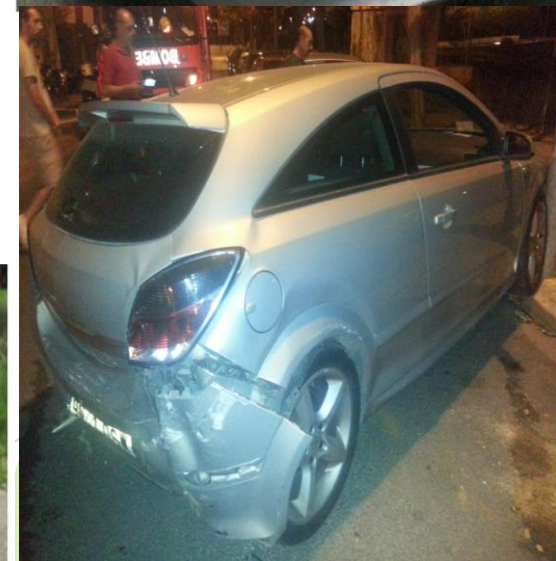
SGD (Stochastic Gradient Descent)

A cada lote de imagens, ajusta os pesos na direção que reduz o erro da rede.

Space warping

Cada camada aplica $h = \text{ReLU}(Wx)$, deformando o espaço dos dados até que classes antes misturadas fiquem separáveis por uma linha reta.

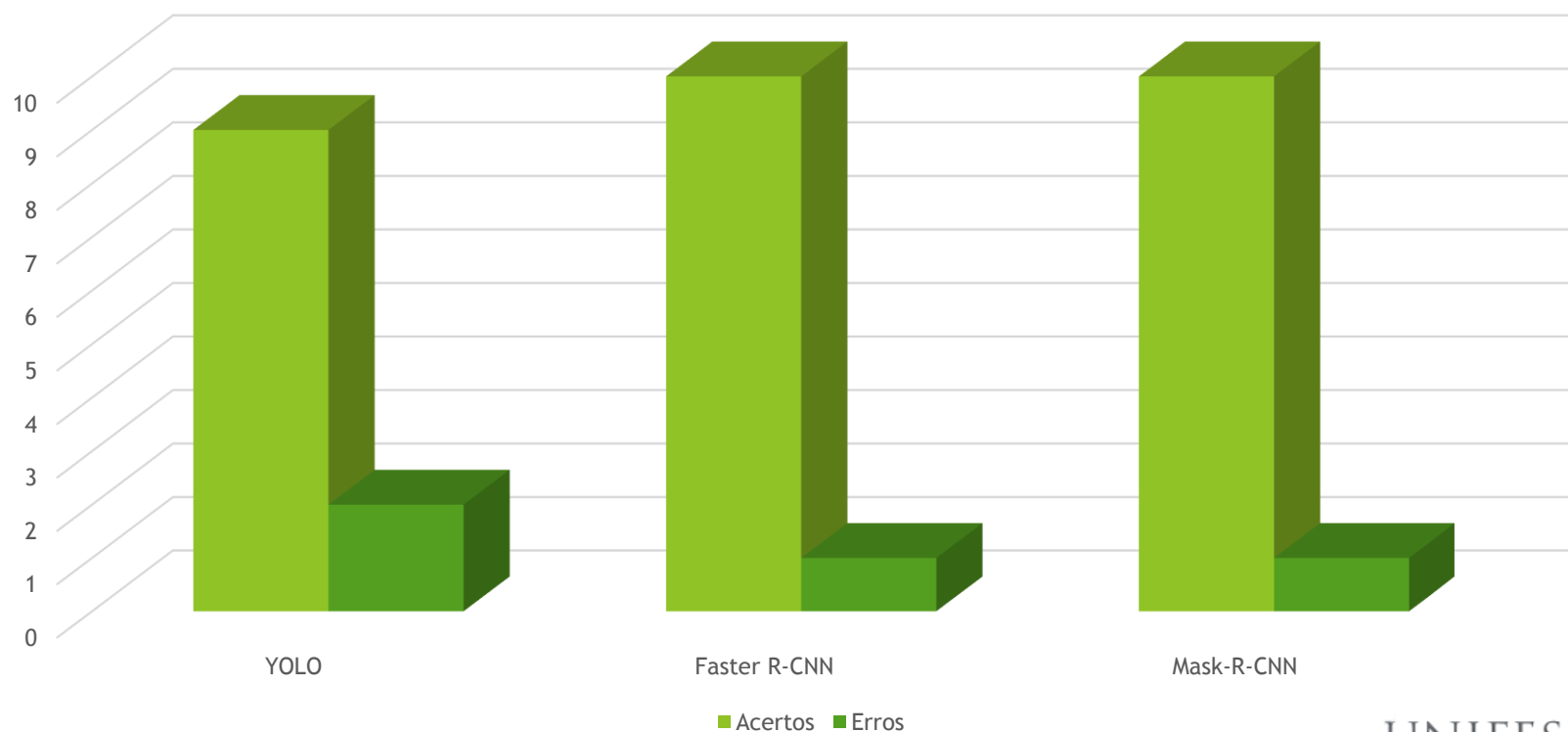
$$h = \text{ReLU}(Wx) = \max(0, Wx)$$



REDES NEURAIS

Rede	Acertos	Erros
YOLO	9	2
Faster R-CNN	10	1
Mask-R-CNN	10	1

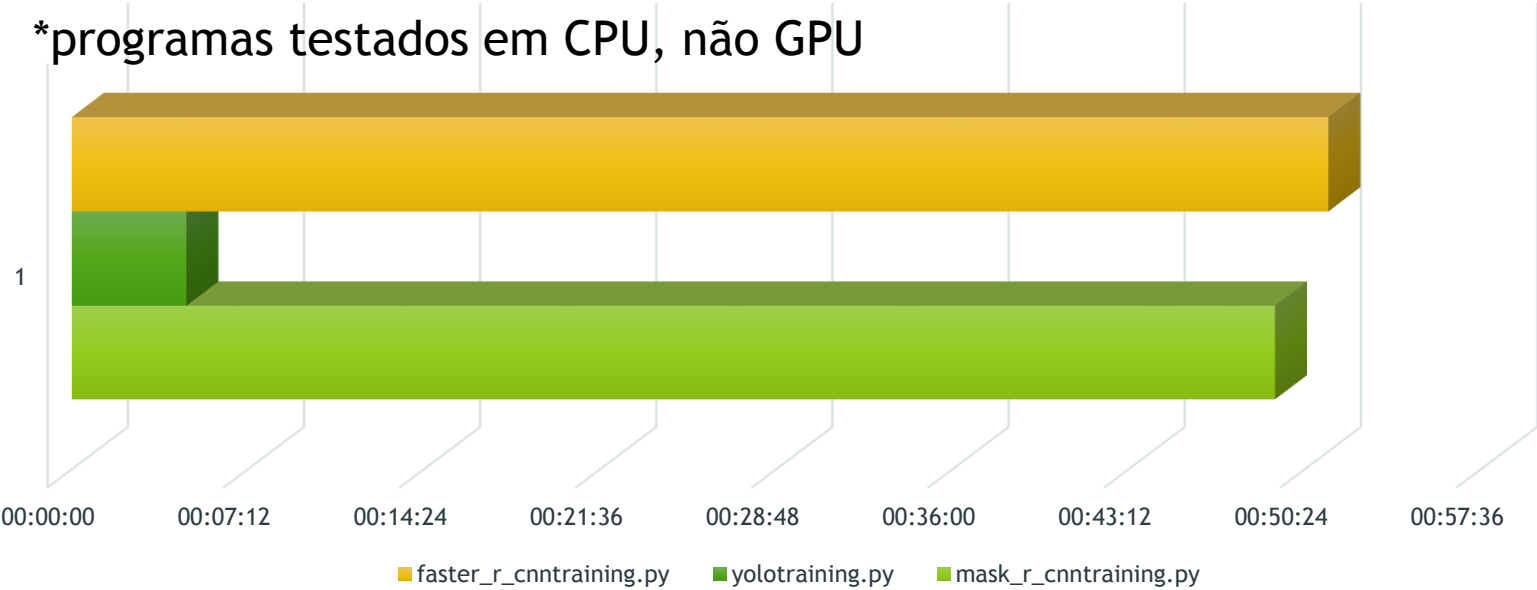
Comparação de Redes



RESULTADOS

TEMPO DE EXECUÇÃO DOS CÓDIGOS

*programas testados em CPU, não GPU



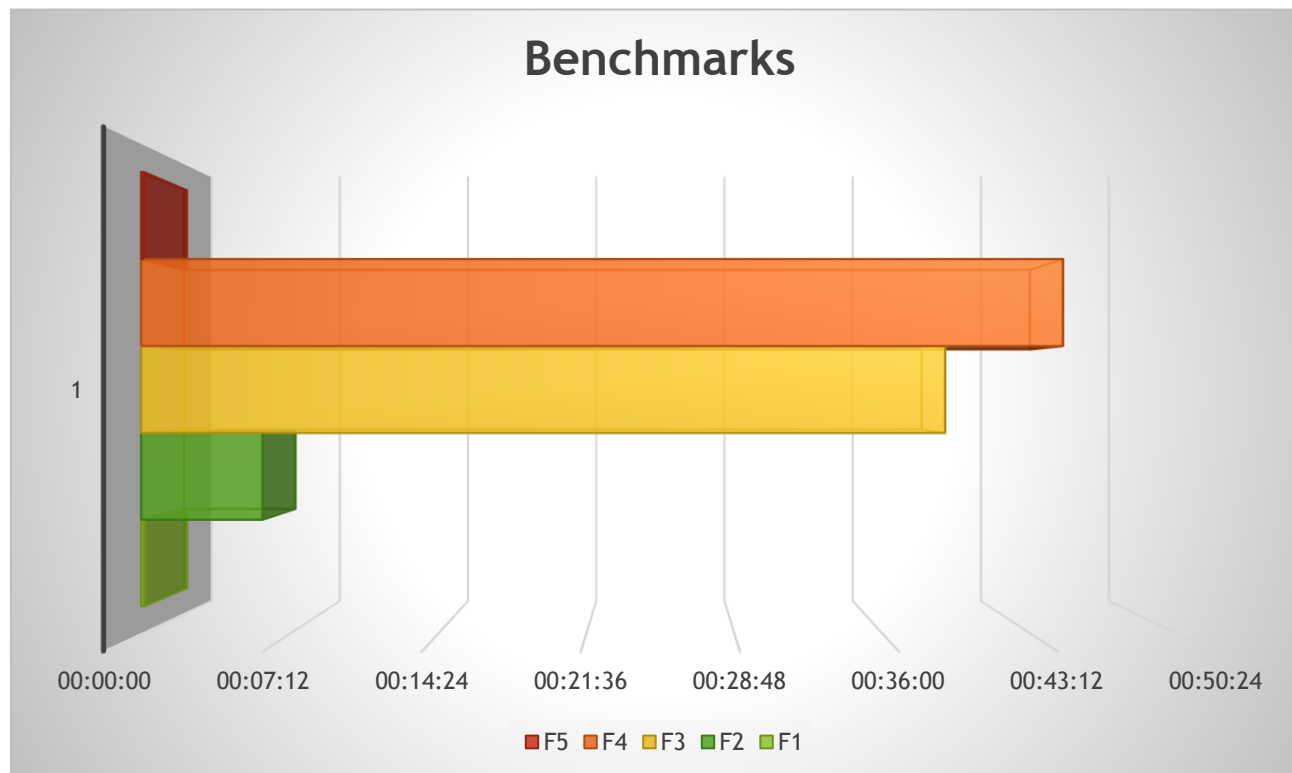
NOME DO CÓDIGO	TEMPO
mask_r_cnnttraining.py	49m11s
yolotraining.py	4m41s
faster_r_cnnttraining.py	51m22s

SCRIPTS AUXILIARES	STATUS
datareading.py	EXECUTADO (7s)
comparacao.py	EM DESENVOLVIMENTO
main.py	EM DESENVOLVIMENTO

FUNÇÕES DO CÓDIGO MAIN

Ferramentas de Registro das Estatísticas:

- Benchmarks: PyCharm + OBS Studio + Relógio (Windows 11)
- Apresentação e Planilhas: Excel + PowerPoint 2016 (Windows 8.1)



BENCHMARKS DAS FUNÇÕES	TEMPO
1 - Exploração do Dataset	10s
2 - Treinar YoloV8	5min54s
3 - Treinar Faster R-CNN	39min01s
4 - Treinar o Mask R-CNN	44min44s
5 - Comparar os 3 Modelos	~9s

CONCLUSÃO

Não há nenhum vencedor único

O melhor modelo depende da métrica priorizada

- **Recall:** o YOLOv8 vence (83.3%) — encontra a maior parte dos danos reais, ao custo de muitos falsos positivos (44 detecções para 24 danos reais).
- **Precision:** o Faster R-CNN vence (100%) — toda detecção que ele faz está correta, mas é conservador e deixa passar quase metade dos danos reais.
- **F1 (equilíbrio geral):** o Faster R-CNN vence (70.3%) — o melhor compromisso entre não gerar alarmes falsos e ainda encontrar a maioria dos danos, seguido de perto pelo Mask R-CNN (65.1%).

Código disponível em: <https://github.com/CaueGhost/cardamage-sar>

Cauê Menezes — UNIFESP 2026